1.1 Pembersihan Teks (Text Cleaning)

Pembersihan teks meliputi serangkaian transformasi untuk menghilangkan noise dan menormalkan representasi teks. Berikut adalah komponen utamanya.

1.1.1 Tokenisasi (Tokenization)

Tokenisasi adalah proses memecah string teks menjadi unit-unit yang lebih kecil yang disebut token. Token dapat berupa kata, sub-kata (subword), atau karakter, tergantung pada granularitas yang dibutuhkan.

- **Word tokenization:** memecah kalimat menjadi kata berdasarkan whitespace dan tanda baca. Contoh: *"Saya makan nasi."* → *["Saya", "makan", "nasi", "."]*
- **Subword tokenization:** memecah kata menjadi unit yang lebih kecil (BPE, WordPiece, SentencePiece). Berguna untuk menangani out-of-vocabulary (OOV) words.
- **Sentence tokenization:** memecah dokumen menjadi kalimat-kalimat berdasarkan tanda baca dan aturan linguistik.

Implementasi cepat dengan NLTK:

```
from nltk.tokenize import word_tokenize, sent_tokenize
tokens = word_tokenize("Saya belajar NLP hari ini.")
# Output: ['Saya', 'belajar', 'NLP', 'hari', 'ini', '.']
```

1.1.2 Lowercasing

Pengubahan teks menjadi huruf kecil bertujuan untuk menyamakan kata dengan kapitalisasi berbeda agar dianggap identik oleh model. Tanpa lowercasing, kata "Indonesia" dan "indonesia" akan dipetakan ke vektor yang berbeda.

```
text = "Saya Belajar NLP".lower()  # "saya belajar nlp"
```

Catatan penting: lowercasing tidak selalu disarankan untuk task seperti Named Entity Recognition (NER) karena kapitalisasi adalah sinyal penting untuk mengidentifikasi nama entitas.

1.1.3 Penghapusan Stopword (Stopword Removal)

Stopword adalah kata-kata yang sangat sering muncul namun memiliki muatan informasi yang rendah, seperti "yang", "di", "dan", "ini" dalam Bahasa Indonesia, atau "the", "is", "at" dalam Bahasa Inggris. Penghapusan stopword mengurangi dimensi fitur dan membantu model fokus pada kata bermakna.

```
from nltk.corpus import stopwords
stop_id = set(stopwords.words('indonesian'))
filtered = [w for w in tokens if w.lower() not in stop_id]
```

Peringatan: untuk task seperti analisis sentimen, kata seperti "tidak" atau "bukan" sangat penting; jangan hapus stopword secara membabi buta tanpa mempertimbangkan konteks task.

1.1.4 Penghapusan Karakter Khusus dan Tanda Baca

Karakter khusus seperti emoji, simbol HTML, URL, mention, hashtag, dan tanda baca biasanya dihapus menggunakan regular expression (regex).

```
import re
text = re.sub(r'http\S+', '', text)        # hapus URL
text = re.sub(r'@\w+', '', text)           # hapus mention
text = re.sub(r'[^a-zA-Z0-9\s]', '', text)  # hapus non-alfanumerik
text = re.sub(r'\s+', ' ', text).strip()   # normalisasi whitespace
```

1.2 Stemming

Stemming adalah proses memotong kata ke bentuk akar (root form) dengan menghilangkan imbuhan (afiks) seperti awalan, sisipan, akhiran, dan konfiks. Tujuannya adalah agar kata-kata yang berasal dari akar yang sama dipetakan ke representasi yang seragam, sehingga mengurangi sparsity vektor fitur.

1.2.1 Konsep Dasar

- **Input:** kata berimbuhan, contoh "berlarian", "pelarian", "melarikan".
- **Output:** akar kata, yaitu "lari".
- **Pendekatan:** berbasis aturan (rule-based) yang mendefinisikan pola imbuhan, atau berbasis kamus untuk validasi.

1.2.2 Stemming vs Lemmatization

Keduanya bertujuan menormalkan kata, namun memiliki pendekatan dan hasil yang berbeda.

Aspek	Stemming	Lemmatization
Pendekatan	Aturan pemotongan afiks	Analisis morfologis dengan kamus (lemma)
Kecepatan	Cepat	Lebih lambat
Hasil	Bisa bukan kata valid (contoh: "studies" → "studi")	Selalu kata valid (contoh: "studies" → "study")
Konteks	Tidak mempertimbangkan POS tag	Mempertimbangkan POS tag dan konteks
Pustaka populer	Porter, Snowball, Sastrawi (ID)	WordNetLemmatizer (NLTK), spaCy

1.2.3 Implementasi Stemming

Untuk Bahasa Inggris, Porter Stemmer adalah algoritma klasik:

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
ps.stem('running')   # 'run'
ps.stem('studies')   # 'studi'
```

Untuk Bahasa Indonesia, gunakan Sastrawi yang menerapkan algoritma Nazief-Adriani:

```
from Sastrawi.Stemmer.StemmerFactory import StemmerFactory
stemmer = StemmerFactory().create_stemmer()
stemmer.stem('berlarian') # 'lari'
stemmer.stem('mempersiapkan')  # 'siap'
```

1.3 Pipeline Pra-pemrosesan Standar

Urutan tahapan yang umum dilakukan dalam pipeline pra-pemrosesan adalah sebagai berikut.

1. Lowercasing seluruh teks.
2. Penghapusan URL, mention, hashtag, dan elemen non-tekstual lainnya.
3. Penghapusan tanda baca dan karakter khusus.
4. Tokenisasi menjadi kata.
5. Penghapusan stopword.
6. Stemming atau lemmatization.
7. Output: list token bersih yang siap diproses tahap ekstraksi fitur.

Bab 2 — Ekstraksi Fitur

Setelah teks dibersihkan, langkah berikutnya adalah mengubah teks menjadi representasi numerik (vektor) agar dapat diproses oleh algoritma machine learning. Tahap ini disebut feature extraction atau text vectorization.

2.1 Bag-of-Words (BoW)

Bag-of-Words memetakan setiap dokumen menjadi vektor numerik yang nilainya merepresentasikan frekuensi kemunculan kata dalam dokumen tersebut. Urutan kata diabaikan; yang dipertahankan hanyalah informasi tentang kata apa saja yang muncul dan berapa kali.

2.1.1 Cara Kerja

8. Bangun kosakata (vocabulary) dari seluruh kata unik di korpus.
9. Untuk setiap dokumen, hitung frekuensi setiap kata dalam vocabulary.
10. Representasikan dokumen sebagai vektor berukuran |V|, di mana |V| adalah ukuran vocabulary.

2.1.2 Ilustrasi

Diberikan dua dokumen:

- D1: "saya suka kopi"
- D2: "saya suka teh dan kopi"

Vocabulary: `[saya, suka, kopi, teh, dan]`

Dokumen	saya	suka	kopi	teh	dan
D1	1	1	1	0	0
D2	1	1	1	1	1

2.1.3 Implementasi

```
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(corpus)
print(vectorizer.get_feature_names_out())
```

2.1.4 Kelebihan dan Kekurangan

- **Kelebihan:** sederhana, mudah diimplementasikan, dan baseline yang kuat untuk task klasifikasi sederhana.
- **Kekurangan:** vektor sangat sparse untuk korpus besar, urutan kata hilang, dan tidak mempertimbangkan importance kata.

2.2 N-Grams

N-Gram adalah sekuen n token berurutan yang diambil dari teks. Penggunaan n-gram memungkinkan model menjaga sebagian urutan dan konteks lokal yang hilang pada BoW murni.

2.2.1 Jenis-jenis N-Gram

N	Nama	Contoh dari kalimat "saya suka NLP"
1	Unigram	["saya", "suka", "NLP"]
2	Bigram	[("saya", "suka"), ("suka", "NLP")]
3	Trigram	[("saya", "suka", "NLP")]

2.2.2 Manfaat dan Trade-off

- **Manfaat:** menangkap konteks lokal seperti "tidak bagus" yang artinya berbeda dengan "bagus" pada unigram.
- **Trade-off:** ukuran vocabulary tumbuh eksponensial, dan banyak n-gram muncul sangat jarang sehingga menyebabkan data sparsity.
- **Praktik umum:** kombinasi unigram + bigram (range (1, 2)) sering memberikan keseimbangan terbaik antara coverage dan sparsity.

2.2.3 Implementasi

```
vectorizer = CountVectorizer(ngram_range=(1, 2))  # unigram + bigram
X = vectorizer.fit_transform(corpus)
```

- **TF-IDF tinggi:** term sering muncul di dokumen ini namun jarang di dokumen lain → kata kunci spesifik.
- **TF-IDF rendah:** term yang umum di mana-mana (mendekati stopword) → bobotnya kecil.

2.3.3 Contoh Perhitungan

Misalkan korpus terdiri dari 3 dokumen, dan term "kopi" muncul di 2 dokumen. Pada dokumen D1 yang memiliki 10 token, kata "kopi" muncul 2 kali.

- TF(kopi, D1) = 2/10 = 0.2
- IDF(kopi) = log(3/2) ≈ 0.176
- TF-IDF(kopi, D1) ≈ 0.2 × 0.176 = 0.0352

2.3.4 Implementasi

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(ngram_range=(1, 2), max_features=5000)
X = tfidf.fit_transform(corpus)
```

2.4 Perbandingan Singkat

Metode	Mempertahankan Urutan	Pembobotan	Sparsity
BoW	Tidak	Frekuensi mentah	Tinggi
N-Gram	Sebagian (lokal)	Frekuensi n-gram	Sangat tinggi
TF-IDF	Tidak (kecuali + n-gram)	TF × IDF	Tinggi tapi terbobot

Bab 3 — Pemodelan Bahasa

Language Model (LM) adalah model probabilistik yang memberikan probabilitas pada sekuen kata. Tujuan utamanya adalah memprediksi kata berikutnya berdasarkan konteks sebelumnya, atau memberikan skor likelihood pada suatu kalimat.

3.1 Konsep N-Gram Language Model

N-Gram LM menghitung probabilitas suatu kalimat dengan mendekomposisinya menjadi probabilitas bersyarat antar kata. Probabilitas penuh suatu kalimat $w_1, w_2, ..., w_n$ dirumuskan dengan chain rule:

$$P(w_1, w_2, ..., w_n) = \Pi P(w_i \mid w_1, ..., w_{i-1})$$

3.1.1 Markov Assumption

Menghitung probabilitas dengan memperhitungkan seluruh history sebelumnya tidak praktis karena masalah sparsity. Markov assumption menyederhanakan dengan mengasumsikan bahwa probabilitas kata berikutnya hanya bergantung pada n-1 kata sebelumnya saja.

- **Unigram (n=1):** $P(w_i)$ — independen sepenuhnya, tidak mempertimbangkan konteks.
- **Bigram (n=2):** $P(w_i \mid w_{i-1})$ — bergantung pada satu kata sebelumnya.
- **Trigram (n=3):** $P(w_i \mid w_{i-2}, w_{i-1})$ — bergantung pada dua kata sebelumnya.

3.1.2 Estimasi Probabilitas dengan MLE

Dengan Maximum Likelihood Estimation, probabilitas bigram dihitung dari rasio frekuensi:

$$P(w_i \mid w_{i-1}) = count(w_{i-1}, w_i) \,/\, count(w_{i-1})$$

Contoh: dari korpus, jika count("saya makan") = 30 dan count("saya") = 100, maka:

$$P(makan \mid saya) = 30 / 100 = 0.30$$

3.1.3 Smoothing

MLE memberi probabilitas nol untuk n-gram yang tidak pernah muncul di data training. Ini bermasalah karena akan membuat probabilitas seluruh kalimat menjadi nol. Solusi umum:

- **Laplace (Add-One) Smoothing:** menambahkan 1 ke setiap count.

$$P(w_i \mid w_{i-1}) = (count(w_{i-1}, w_i) + 1) \,/\, (count(w_{i-1}) + |V|)$$

- **Add-k Smoothing:** generalisasi dengan menambahkan k < 1.
- **Kneser-Ney Smoothing:** teknik canggih yang sering memberi performa terbaik untuk n-gram LM.
- **Backoff dan Interpolation:** kombinasikan probabilitas dari unigram, bigram, trigram untuk mengatasi sparsity.

3.1.4 Evaluasi: Perplexity

Perplexity adalah metrik standar untuk mengevaluasi LM. Semakin rendah nilainya, semakin baik model dalam memprediksi data uji.

$$PP(W) = P(w_1, w_2, ..., w_n)^{(-1/n)}$$

Secara intuitif, perplexity adalah rata-rata cabang efektif yang dihadapi model di setiap langkah prediksi.

3.2 Fungsi Utama Language Model

Language Model menjadi komponen mendasar di banyak aplikasi NLP modern.

3.2.1 Smart Reply dan Auto-completion

Sistem seperti smart reply pada Gmail atau auto-completion keyboard di smartphone menggunakan LM untuk memprediksi kata atau frasa selanjutnya berdasarkan konteks yang sudah diketik pengguna.

3.2.2 Pengenalan Suara (Automatic Speech Recognition)

Pada ASR, LM digunakan untuk memilih hipotesis transkripsi yang paling probabel di antara beberapa kandidat yang dihasilkan acoustic model. LM membantu disambiguasi kata homofon seperti "rok" vs "rock".

3.2.3 Terjemahan Mesin (Machine Translation)

Pada statistical machine translation klasik (model noisy channel), LM bahasa target memastikan output terjemahan terdengar natural dan gramatikal di bahasa target.

3.2.4 Aplikasi Lain

- Spelling correction dan grammar checking.
- Optical Character Recognition (OCR) — disambiguasi karakter mirip.
- Information retrieval (query likelihood model).
- Text generation untuk chatbot, summarization, dan creative writing.

3.3 Keterbatasan N-Gram LM

- **Sparsity:** banyak n-gram tidak pernah muncul di training, walau valid.
- **Konteks pendek:** hanya mempertimbangkan n-1 kata sebelumnya, sulit menangkap dependensi jangka panjang.
- **Storage:** ukuran tabel n-gram tumbuh eksponensial dengan n.

Keterbatasan ini menjadi motivasi pengembangan neural language model seperti RNN, LSTM, dan Transformer (GPT, BERT) yang dapat menangkap konteks lebih luas dan generalisasi lebih baik.

Bab 4 — Analisis Tata Bahasa dan Entitas

Bab ini membahas dua task fundamental NLP: Part-of-Speech (POS) Tagging untuk melabeli kelas kata, dan Named Entity Recognition (NER) untuk mengekstrak entitas seperti nama orang, lokasi, dan tanggal dari teks.

4.1 POS Tagging (Part-of-Speech Tagging)

POS Tagging adalah proses memberi label kelas kata (noun, verb, adjective, adverb, dst.) pada setiap token dalam kalimat. Output ini menjadi fitur dasar untuk task downstream seperti parsing, NER, dan information extraction.

Contoh: "Saya membaca buku." → [("Saya", PRON), ("membaca", VERB), ("buku", NOUN), (".", PUNCT)]

4.1.1 Pendekatan Rule-Based

Pendekatan ini mengandalkan kamus dan aturan linguistik manual untuk menentukan tag suatu kata berdasarkan morfologi dan konteks sintaktiknya.

- **Kelebihan:** transparan dan dapat dikontrol secara eksplisit oleh ahli linguistik.
- **Kekurangan:** rapuh terhadap variasi bahasa, sulit di-maintain saat aturan bertambah dan tidak skalabel untuk bahasa dengan morfologi kompleks.

4.1.2 Pendekatan Transformation-Based (Brill Tagger)

Brill Tagger adalah pendekatan hybrid yang awalnya memberikan tag default berdasarkan frekuensi, lalu menerapkan rangkaian aturan transformasi yang dipelajari otomatis dari data berlabel untuk mengoreksi tag awal tersebut.

- **Cara kerja:** learn rules of the form "ubah tag X menjadi Y jika kondisi konteks ... terpenuhi".
- **Kelebihan:** aturan yang dihasilkan dapat dibaca manusia, lebih akurat dari rule based murni.

4.1.3 Pendekatan Statistik: Hidden Markov Model (HMM)

HMM memodelkan sekuen tag sebagai hidden states dan kata sebagai observations. Model ini memilih sekuen tag yang memaksimalkan probabilitas gabungan.

$$\underset{t}{argmax}\ T\,\Pi\, P(T \mid W) = \underset{t}{argmax}\ T\,\Pi\, P(t_i \mid t_{i-1}) \times P(w_i \mid t_i)$$

Dua probabilitas utama:

- **Transition probability:** $P(t_i \mid t_{i-1})$ — peluang transisi antar tag.
- **Emission probability:** $P(w_i \mid t_i)$ — peluang menghasilkan kata w_i dari tag t_i.

Dekoding (mencari sekuen tag terbaik) dilakukan dengan algoritma **Viterbi** yang efisien menggunakan dynamic programming.

4.1.4 Pendekatan Modern: Neural Tagger

Pendekatan modern menggunakan BiLSTM-CRF atau Transformer-based tagger (BERT) yang memberi hasil state-of-the-art. Model ini mempelajari representasi kontekstual dan mampu menangani out-of-vocabulary lebih baik.

4.2 NER (Named Entity Recognition)

NER adalah task mengekstrak dan mengklasifikasikan entitas bernama (named entities) dalam teks ke kategori-kategori predefined seperti PERSON, ORGANIZATION, LOCATION, DATE, MONEY, dan PERCENT.

Contoh: "Joko Widodo lahir di Surakarta pada 21 Juni 1961."

- **Joko Widodo** → PERSON
- **Surakarta** → LOCATION
- **21 Juni 1961** → DATE

4.2.1 Pendekatan Berbasis Kamus (Dictionary-Based / Gazetteer)

Sistem mencocokkan token pada teks dengan daftar entitas predefined (gazetteer). Contohnya, daftar nama kota Indonesia digunakan untuk mengekstrak entitas LOCATION.

- **Kelebihan:** akurasi tinggi untuk entitas yang ada di kamus.
- **Kekurangan:** tidak dapat mengenali entitas baru yang tidak ada di kamus, dan rentan ambiguitas ("Solo" sebagai kota vs nama orang).

4.2.2 Pendekatan Berbasis Aturan

Menggunakan pola regex dan aturan linguistik. Contoh aturan: "jika dua kata berurutan diawali huruf kapital dan didahului kata 'di', maka kandidat LOCATION".

- **Kelebihan:** efektif untuk pola yang sangat regular seperti tanggal atau mata uang.
- **Kekurangan:** sulit diperluas untuk entitas yang variatif.

4.2.3 Pendekatan Machine Learning

Memodelkan NER sebagai sequence labeling task dengan skema BIO/BIOES.

- **Skema BIO:** `B-PER` = beginning of person, `I-PER` = inside of person, `O` = outside (bukan entitas).
- **Algoritma klasik:** Conditional Random Fields (CRF), HMM, MaxEnt.
- **Algoritma modern:** BiLSTM-CRF, BERT-NER, IndoBERT-NER untuk Bahasa Indonesia.

7.1.2 Implementasi

```
from wordcloud import WordCloud
from collections import Counter
from nltk import ngrams
import matplotlib.pyplot as plt

bigrams = list(ngrams(tokens, 2))
freq = Counter([' '.join(bg) for bg in bigrams])

wc = WordCloud(width=1200, height=600,
background_color='white').generate_from_frequencies(freq)
plt.imshow(wc, interpolation='bilinear')
plt.axis('off'); plt.show()
```

7.2.3 Implementasi Singkat

```
# library populer: holoviews, plotly, atau bokeh
import holoviews as hv
hv.extension('bokeh')

data: list of (source, target, value)
chord = hv.Chord(data)
chord.opts(labels='entity', node_color='entity', edge_color='source')
```

```
import plotly.express as px

fig = px.scatter(df, x='x', y='y', size='count', color='category',
                 hover_name='label', size_max=40)
fig.update_layout(showlegend=True)
fig.show()
```

7.4 Panduan Memilih Visualisasi

Tujuan Analisis	Visualisasi yang Direkomendasikan
Identifikasi kata/frasa dominan	N-Gram Word Cloud
Eksplorasi relasi antar entitas	Chord Diagram
Perbandingan frekuensi antar kategori	Packed Bubble Chart
Distribusi topik per dokumen	Stacked Bar Chart atau Heatmap
Evolusi topik antar waktu	Stream Graph atau Line Chart
Co-occurrence matrix	Heatmap atau Chord Diagram

7.5 Prinsip Umum Visualisasi Teks

- **Lakukan pra-pemrosesan dengan baik.** Visualisasi tanpa pra-pemrosesan akan didominasi noise seperti stopword.
- **Pilih visualisasi yang tepat untuk pesan.** Setiap chart memiliki kekuatan dan keterbatasan.
- **Berikan konteks.** Selalu sertakan title, axis label, legend, dan caption yang jelas.
- **Hindari chartjunk.** Elemen dekoratif yang tidak menambah informasi mengurangi keterbacaan.
- **Pertimbangkan aksesibilitas.** Gunakan colorblind-friendly palette dan kontras yang cukup.

A. Pipeline NLP End-to-End

26. Pengumpulan korpus mentah.
27. Pra-pemrosesan: tokenisasi, lowercasing, stopword removal, stemming/lemmatization (Bab 1).
28. Ekstraksi fitur: BoW, TF-IDF, atau embedding (Bab 2).
29. Pemodelan: language model, classifier, atau topic model (Bab 3, 5, 6).
30. Analisis linguistik: POS tagging dan NER bila dibutuhkan (Bab 4).
31. Evaluasi: metrik task-specific (Bab 5, 6).
32. Visualisasi dan penyajian (Bab 7).

B. Daftar Pustaka Python yang Sering Digunakan

Pustaka	Kegunaan Utama
NLTK	Tokenisasi, POS tagging, stopword, stemming (klasik)
spaCy	Pipeline NLP industri: tokenisasi, POS, NER, parser
Sastrawi	Stemming Bahasa Indonesia (Nazief-Adriani)
scikit-learn	BoW, TF-IDF, classifier ML tradisional, evaluasi
gensim	LDA, LSA, Word2Vec, FastText, coherence
BERTopic	Topic modeling berbasis embedding
transformers (HF)	Model BERT, GPT, T5, fine-tuning
wordcloud	Visualisasi word cloud
plotly / holoviews	Chord diagram, packed bubble, visualisasi interaktif
matplotlib / seaborn	Visualisasi statis dasar

C. Rumus Penting

- **TF-IDF:** $TF\text{-}IDF(t,d) = TF(t,d) \times \log(N/df(t))$
- **Bigram MLE:** $P(w_i \mid w_{i-1}) = count(w_{i-1}, w_i) \,/\, count(w_{i-1})$
- **Laplace Smoothing:** $P(w_i \mid w_{i-1}) = (count(w_{i-1}, w_i) + 1) \,/\, (count(w_{i-1}) + |V|)$
- **Accuracy:** $(TP+TN) / (TP+TN+FP+FN)$
- **Precision:** $TP / (TP+FP)$
- **Recall:** $TP / (TP+FN)$
- **F1-Score:** $2 \times Precision \times Recall / (Precision + Recall)$
- **Perplexity:** $exp(-\text{log prob} / N)$
- **LSA:** $A \approx U \times \Sigma \times V^T$ (truncated SVD)
- **NMF:** $A \approx W \times H$, dengan $W \geq 0$ dan $H \geq 0$